



UNIVERSIDADE FEDERAL DO MARANHÃO
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIAS
ENGENHARIA DA COMPUTAÇÃO

Prof. Dr. Denis Fabricio Sousa de Sá

Emanuel Lopes Silva

**Desenvolvimento de Ferramenta Computacional
para Análise Estrutural de Sistemas LIT:
Extração de Polos, Zeros e Lugar das Raízes**

São Luís
2026

Emanuel Lopes Silva

**Desenvolvimento de Ferramenta Computacional para
Análise Estrutural de Sistemas LIT:
Extração de Polos, Zeros e Lugar das Raízes**

Relatório técnico apresentado à disciplina
de Engenharia de Controle (Engenharia da
Computação - UFMA) seguindo a estrutura
de análise teórica e produção experimental.

Orientador: Prof. Dr. Denis Fabricio Sousa de Sá

São Luís
2026

Resumo

Este documento descreve o desenvolvimento e a implementação de uma arquitetura de software focada na extração, mapeamento e análise estrutural de singularidades (polos e zeros) em Sistemas Lineares e Invariantes no Tempo (LIT). Desenvolvida nativamente em linguagem Python, a ferramenta emprega o método iterativo de Durand-Kerner para o cálculo global de raízes complexas, assegurando alta precisão sem a necessidade de dependências externas. O trabalho detalha a modelagem algébrica de funções de transferência, a conversão para o espaço de estados, o traçado parametrizado do Lugar Geométrico das Raízes com restrição de amortecimento (ζ). Adicionalmente, aborda-se a integração de rotinas heurísticas na interface gráfica, criando um ambiente computacional altamente confiável e didático para o estudo da Teoria de Controle.

Palavras-chave: Sistemas LIT; Teoria de Controle; Polos e Zeros; Lugar das Raízes; Python.

Sumário

1	INTRODUÇÃO	4
1.1	Contextualização e Justificativa	4
1.2	Objetivos	5
1.2.1	Objetivo Geral	5
1.2.2	Objetivos Específicos	5
2	DETALHAMENTO TEÓRICO E ALGORÍTMICO	7
2.1	math_utils.py: Núcleo Algébrico Fundamental	7
2.2	transfer_function.py: Processo de Abstração	9
2.3	root_locus.py: Geometria Dinâmica e Mapeamento Paramétrico	10
2.4	plotting.py: Isometria e Exibição Analítica Especializada	11
2.5	analysis_explainer.py: Diagnóstico Heurístico e Cancelamentos	13
2.6	gif_tools.py: Geração Didática de GIF (completamente opcional)	14
2.7	time_response.py: Validação Temporal Fundamental	15
2.8	input_response.py: Rastreamento Físico Arbitrário	16
2.9	gui.py: Interface de Operação Baseada em Eventos	18
3	IMPLEMENTAÇÃO E VALIDAÇÃO	20
3.1	Fluxo de Operação e Ciclo de Vida do Sistema	20
3.2	Galeria de Funcionalidades e Validação Visual	21
3.3	Processo de Robustez	22
4	CONCLUSÃO	26
	REFERÊNCIAS	27

1 Introdução

A teoria de Sistemas Lineares e Invariantes no Tempo (LIT) constitui um dos pilares fundamentais da engenharia moderna, com aplicações consolidadas nas áreas de automação, processamento de sinais, telecomunicações e controle de processos industriais. A análise do comportamento desses sistemas, tanto no domínio do tempo quanto no domínio da frequência, permite prever o desempenho, projetar controladores robustos e garantir a estabilidade de plantas dinâmicas complexas(DORF; BISHOP, 2011; OGATA, 2010).

Tradicionalmente, o ambiente acadêmico e industrial recorre a ferramentas comerciais de alto custo ou a ecossistemas computacionais de código aberto consolidados para realizar tais análises. No entanto, o uso dessas ferramentas frequentemente assume o caráter de uma "caixa preta", ocultando os algoritmos numéricos subjacentes e os rigorosos passos matemáticos intermediários. Essa abstração, embora eficiente para fins de desenvolvimento industrial, pode comprometer a compreensão intuitiva e o aprendizado aprofundado por parte de estudantes de engenharia e ciências exatas.

Diante desse cenário, surge a oportunidade de desenvolver ferramentas de cunho estritamente pedagógico que priorizem a transparência algorítmica. O presente trabalho documenta o desenvolvimento de uma biblioteca computacional de análise linear desenvolvida inteiramente do zero, integrada a um ambiente interativo em Python puro capaz de gerar relatórios dedutivos completos, animações didáticas de cálculo e uma interface visual de operação, com o propósito de demonstrar detalhadamente o passo a passo matemático envolvido na análise e simulação de sistemas LIT.

1.1 Contextualização e Justificativa

A modelagem de sistemas LIT na forma de Funções de Transferência $G(s) = N(s)/D(s)$ exige a manipulação de polinômios complexos para a determinação de parâmetros vitais de estabilidade, como polos e zeros. Softwares tradicionais utilizam rotinas altamente otimizadas compiladas em linguagens de baixo nível para realizar tais cálculos(GOLNARAGHI; KUO, 2010; NISE, 2012).

Este projeto justifica-se pela necessidade de uma plataforma didática onde o estudante possa rastrear como métodos numéricos puros realizam essa transição. Ao implementar rotinas manuais de aritmética polinomial, respostas temporais e mapeamentos frequenciais sem depender de pacotes de cálculo científico (como `SciPy` ou `control`), o código-fonte desenvolvido torna-se uma extensão direta do livro-texto de controle clássico, servindo como uma ponte entre a matemática abstrata e a computação aplicada.

1.2 Objetivos

Nesta seção, delimitam-se os propósitos deste trabalho, divididos entre o escopo amplo da aplicação desenvolvida do zero e as metas específicas de engenharia de software e implementação matemática.

1.2.1 Objetivo Geral

O objetivo geral deste trabalho consiste no projeto, desenvolvimento integral do zero e validação de uma biblioteca modular em Python puro, dotada de uma interface gráfica de usuário (GUI), voltada para a simulação temporal, análise de resposta em frequência, mapeamento de estabilidade e geração de explicações textuais detalhadas passo a passo de sistemas dinâmicos lineares e invariantes no tempo (LIT).

1.2.2 Objetivos Específicos

Para o alcance do objetivo geral, estabelecem-se as seguintes metas específicas de desenvolvimento:

- Desenvolver um motor de álgebra e aritmética polinomial puro capaz de realizar operações estruturais de alinhamento, soma, subtração, convolução e extração de raízes complexas utilizando o método iterativo de Durand-Kerner;
- Implementar uma classe de representação simbólica para funções de transferência contínuas que gerencie a normalização automática de coeficientes líderes e realize álgebra de blocos, abrangendo associações em série, paralelo e malhas de realimentação unitária ou genérica;
- Formular um algoritmo de conversão do modelo polinomial da função de transferência diretamente para a representação em espaço de estados na forma canônica controlável, gerando as matrizes dinâmicas de sistema A , B , C e D ;
- Aplicar o método de integração numérica de Runge-Kutta de 4ª ordem (RK4) para solucionar iterativamente as equações de estado e simular respostas a sinais de teste clássicos como degrau unitário e impulso ou funções de entrada discreta arbitrárias;
- Estruturar rotinas de varredura em frequência para cálculo de resposta complexa $G(j\omega)$, viabilizando a geração nativa de dados para diagramas de Bode (magnitude em dB e fase em graus), diagramas polar/Nyquist e trajetórias de malha fechada via Lugar Geométrico das Raízes (Root Locus);
- Construir um subsistema de explicabilidade capaz de auditar a função de transferência, detectar cancelamentos aproximados polo-zero dentro de limites de tolerância

numéricas, mapear multiplicidades e emitir relatórios textuais estruturados passo a passo em linguagem natural;

- Desenvolver ferramentas visuais acessórias para exportação do comportamento dinâmico e passos de cálculo em formatos de imagens estáticas ou animações temporais no formato GIF;
- Projetar uma interface gráfica de usuário interativa e estilizada utilizando a biblioteca nativa `tkinter` para facilitar a inserção dinâmica de parâmetros pelo usuário e a chamada centralizada das funções analíticas.

2 Detalhamento Teórico e Algorítmico

O programa desenvolvido consiste em uma arquitetura de software elaborada especificamente para a extração, mapeamento e análise rigorosa de polos e zeros de sistemas dinâmicos lineares. O escopo do projeto afasta-se de abordagens genéricas em ferramentas comerciais de amplo espectro, concentrando seu núcleo computacional exclusivamente na dissecação exata das propriedades intrínsecas da planta baseadas em suas raízes matemáticas. O foco incide sobre a localização das singularidades no plano complexo e a migração de estabilidade parametrizada via Lugar das Raízes (Root Locus), com ênfase no rastreamento do fator de amortecimento (ζ). Abaixo apresenta-se a explicação técnica, teórica e acadêmica de cada módulo estrutural que compõe a biblioteca, detalhando os alicerces teóricos e as implementações em código-fonte que viabilizam essa extração espacial com alta fidelidade analítica.

2.1 `math_utils.py`: Núcleo Algébrico Fundamental

O módulo `math_utils.py` atua como o motor algébrico basilar do programa, encarregado de solucionar o problema central da manipulação de polinômios e da extração de raízes complexas. Uma premissa fundamental do desenvolvimento foi a de não recorrer a bibliotecas numéricas pré-compiladas para os cálculos de base, garantindo ao operador total transparência e controle absoluto sobre as tolerâncias computacionais, essenciais no estudo das singularidades. No escopo da Teoria de Controle Clássico, a determinação exata das raízes do polinômio característico (denominador) e do numerador é o passo primário e indispensável para encontrar os polos e zeros de qualquer sistema.

Para a manipulação de matrizes polinomiais, o código implementa algoritmos eficientes baseados em listas puras. A multiplicação de polinômios, essencial para fechar malhas ou associar blocos em série, é resolvida por meio da convolução discreta unidimensional. Em paralelo, a avaliação de polinômios em pontos específicos do plano complexo emprega o método de Horner (`polyval`), que minimiza o número de multiplicações necessárias e reduz exponencialmente os erros de arredondamento inerentes à aritmética de ponto flutuante, um fator crítico ao avaliar funções de transferência próximas a polos de alta ordem.

O coração algorítmico do módulo reside na função `polynomial_roots`, que emprega o método iterativo numérico de Durand-Kerner (também conhecido como método de Weierstrass). Trata-se de uma técnica iterativa de convergência global capaz de determinar simultaneamente todas as raízes de um polinômio mônico de grau n . Partindo de sementes complexas distribuídas circularmente no plano (obtidas pelas raízes da unidade), o

algoritmo refina cada aproximação iterativamente, penalizando o passo com as distâncias em relação às outras raízes estimadas. A implementação lida de maneira autônoma com o recorte (*trimming*) e a normalização algébrica para garantir que o coeficiente líder seja estritamente unitário. A robustez do Durand-Kerner oferece notável estabilidade numérica frente a raízes complexas conjugadas e configurações de multiplicidade densa.

Listing 2.1 – Extração de Raízes pelo Método Durand-Kerner (`math_utils.py`)

```

1 def polynomial_roots(coefficients, max_iter=200, tol=1e-12):
2     """Encontra raízes complexas usando o método de Durand-Kerner.
3
4     O algoritmo é robusto o suficiente para as necessidades deste
5     projeto,
6     inclusive para raízes complexas conjugadas e multiplicidades
7     baixas.
8     """
9     polynomial = trim_polynomial(coefficients, tol=tol)
10    degree = len(polynomial) - 1
11    if degree < 1:
12        return []
13    if degree == 1:
14        return [-polynomial[1] / polynomial[0]]
15
16    monic = normalize_polynomial(polynomial, tol=tol)
17    roots = _initial_root_guesses(degree)
18    if not roots:
19        return []
20
21    for _ in range(max_iter):
22        max_delta = 0.0
23        new_roots = roots[:]
24        for i in range(degree):
25            denominator = 1 + 0j
26            for j in range(degree):
27                if i != j:
28                    diff = roots[i] - roots[j]
29                    if abs(diff) < tol:
30                        diff = tol + 0j
31                    denominator *= diff
32
33            value = polyval(monic, roots[i])
34            delta = value / denominator
35            new_roots[i] = roots[i] - delta
36            max_delta = max(max_delta, abs(delta))
37
38    roots = new_roots
39    if max_delta < tol:

```

```

38         break
39
40     return roots

```

2.2 transfer_function.py: Processo de Abstração

A classe `TransferFunction` representa a principal estrutura de dados simbólica do projeto, encarregada de modelar matematicamente a planta. Em sistemas físicos modelados por equações diferenciais lineares e invariantes no tempo (LIT), a transição para o domínio transformado (Laplace ou Z) resulta em uma relação entrada-saída expressa rigorosamente por uma razão racional de polinômios, $G(s) = \frac{N(s)}{D(s)}$. O propósito primário deste módulo é encapsular essa relação para que as singularidades (polos e zeros) possam ser adequadamente isoladas pelo motor matemático.

No nível algorítmico, a classe interage com as operações definidas em `math_utils.py` através da sobrecarga de operadores nativos da linguagem (`__add__`, `__sub__`, `__mul__`, `__truediv__`). Isso permite associações complexas como série, paralelo e realimentação (`feedback`), recomputando dinamicamente os novos polinômios numeradores e denominadores equivalentes da malha fechada. A formulação ativa dessa álgebra racional elimina o trabalho mecânico e previne erros comuns no alinhamento de graus durante as somas de frações parciais.

Outra implementação teórica crítica no módulo é o método `to_state_space()`, que efetua o mapeamento direto do modelo racional contínuo para o espaço de estados na forma canônica controlável. Matematicamente expresso por $\dot{x}(t) = Ax(t) + Bu(t)$ e $y(t) = Cx(t) + Du(t)$, o algoritmo preenche a matriz companheira inferior A invertendo e alocando sistematicamente os coeficientes normalizados do denominador. As matrizes vetoriais B, C, D são deduzidas pela diferença sistemática das ordens polinomiais, consolidando a ponte definitiva entre a abstração simbólica em frequência e a necessidade de integração numérica no domínio do tempo para simulações de validação.

Listing 2.2 – Mapeamento para a Forma Canônica Controlável (`transfer_function.py`)

```

1     def to_state_space(self):
2         """Converte a função de transferência para forma canônica
           controlável."""
3         num = trim_polynomial(self.numerator)
4         den = normalize_polynomial(self.denominator)
5
6         if den == [0.0]:
7             raise ValueError("Denominador inválido.")
8
9         order = len(den) - 1
10        if order < 1:

```

```

11         raise ValueError("A ordem do sistema deve ser pelo menos
12             1.")
13
14     num = [value / self.denominator[0] for value in num]
15
16     if len(num) < order + 1:
17         num = [0.0] * (order + 1 - len(num)) + num
18     elif len(num) > order + 1:
19         raise ValueError("O sistema deve ser próprio ou
20             estritamente próprio.")
21
22     a = den[1:]
23     b = num
24     d = b[0]
25     c_coeffs = [b[order - i] - a[order - i - 1] * d for i in range(
26         order)]
27
28     A = [[0.0 for _ in range(order)] for _ in range(order)]
29     for i in range(order - 1):
30         A[i][i + 1] = 1.0
31     A[-1] = [-value for value in reversed(a)]
32
33     B = [[0.0] for _ in range(order)]
34     B[-1][0] = 1.0
35
36     C = [c_coeffs]
37     D = [[d]]
38     return A, B, C, D

```

2.3 root_locus.py: Geometria Dinâmica e Mapeamento Paramétrico

O módulo `root_locus.py` estende a análise puramente estática das raízes, sendo inteiramente dedicado a mapear a migração contínua das posições dos polos de malha fechada no plano complexo sob a variação escalar e paramétrica de um ganho estrito K . A disposição geométrica e a trajetória dessas raízes ditam inequivocamente a taxa de amortecimento da resposta transiente e os limiares críticos de estabilidade absoluta perante o critério de Routh-Hurwitz.

A formulação matemática implementada dispensa o traçado de assíntotas gráficas aproximadas, optando pela avaliação iterativa e estrita da Equação Característica da malha fechada, deduzida de $1 + K \cdot \frac{N(s)}{D(s)} = 0$, que se reorganiza na forma polinomial canônica $D(s) + K \cdot N(s) = 0$. O script aceita como entrada uma instância da `TransferFunction` e um vetor iterável de ganhos arbitrários.

Para cada degrau escalar do vetor de ganhos, o código instancia um novo polinômio característico fundindo as matrizes do denominador e do numerador ponderado. Este novo polinômio sintético é submetido repetidamente ao solucionador numérico de Durand-Kerner, extraíndo e registrando um quadro vetorial com os polos instantâneos. O algoritmo agrega esses resultados a um grande dicionário em memória que preserva não apenas os polos fechados, mas também os polos e zeros originais de malha aberta, garantindo que as origens topológicas de cada ramo no plano complexo possam ser reconstruídas perfeitamente durante a visualização gráfica.

Listing 2.3 – Iteração e Cálculo do Lugar das Raízes (`root_locus.py`)

```

1 def root_locus(tf, gains):
2     """Calcula as raízes da equação característica para cada ganho
3     K."""
4     if not isinstance(tf, TransferFunction):
5         raise TypeError("tf deve ser uma instância de TransferFunction
6         .")
7
8     roots_data = []
9     open_loop_poles = tf.poles
10    open_loop_zeros = tf.zeros
11
12    den = tf.denominator
13    num = tf.numerator
14
15    for gain in gains:
16        characteristic = add_polynomials(den, [gain * value for value
17        in num])
18        roots = polynomial_roots(characteristic)
19        roots_data.append(
20            {
21                "K": gain,
22                "roots": roots,
23                "poles": open_loop_poles,
24                "zeros": open_loop_zeros,
25            }
26        )
27
28    return roots_data

```

2.4 plotting.py: Isometria e Exibição Analítica Especializada

O módulo `plotting.py` gerencia a formatação visual do sistema, assumindo a responsabilidade crítica de converter as tensões e vetores numéricos de raízes abstratas em visualizações geométricas de alto rigor e padronização através da suíte `Matplotlib`. O

isolamento dessa responsabilidade garante que os módulos matemáticos não possuam acoplamento de estado com variáveis de interface e desenho, servindo unicamente ao mapeamento isométrico.

Dado o escopo focado em singularidades estruturais, o código fornece funções de espalhamento (*scatter plot*) especializadas. A função `plot_pole_zero_map` renderiza as fronteiras de distribuição global no plano complexo, demarcando zeros com "o" e polos com "x". Já a função `plot_pole_zero_zoom` provê o amparo analítico microscópico, essencial quando há presença de raízes de alta proximidade (quase-cancelamentos) ou múltiplas ordens densas, onde a configuração de um "centro" (coordenada complexa paramétrica) e um "*span*" revela detalhamentos que definem o verdadeiro comportamento em baixas frequências.

O principal trunfo visual voltado à engenharia, contudo, é a sub-rotina de traçado paramétrico em `plot_root_locus_with_zeta`. O lugar geométrico comum exhibe apenas dinâmicas em aberto; no entanto, requisitos temporais de sistemas subamortecidos dependem do fator de amortecimento, ζ . Geometricamente, lugares com amortecimento constante formam um feixe de raios lineares a partir da origem estendendo-se ao semiplano esquerdo, governado pelo ângulo intrínseco $\theta = \arccos(\zeta)$. O código computa trigonometricamente essas fronteiras de fase e realiza a injeção dessas linhas tracejadas no gráfico, fornecendo referência visual imediata para o ponto de ganho ótimo de interseção.

Listing 2.4 – Sobreposição Vetorial do Fator de Amortecimento (`plotting.py`)

```

1 def plot_root_locus_with_zeta(roots_data, zeta=None):
2     fig, ax = plot_root_locus(roots_data)
3
4     if zeta is not None:
5         zeta = max(min(float(zeta), 0.999999), -0.999999)
6         angle = acos(zeta)
7         limit = max(
8             1.0,
9             abs(ax.get_xlim()[0]),
10            abs(ax.get_xlim()[1]),
11            abs(ax.get_ylim()[0]),
12            abs(ax.get_ylim()[1]),
13        )
14        x_values = [-limit * cos(angle), 0.0]
15        y_values = [limit * sin(angle), 0.0]
16        ax.plot(x_values, y_values, linestyle="--", linewidth=1.6,
17              color="tab:orange", label=f"    = {zeta:g}")
18        ax.plot(x_values, [-value for value in y_values], linestyle
19             ="--", linewidth=1.6, color="tab:orange")
20        ax.legend(loc="best")
21
22    fig.tight_layout()

```

```
21 return fig, ax
```

2.5 analysis_explainer.py: Diagnóstico Heurístico e Cancelamentos

O módulo `analysis_explainer.py` confere inteligência textual e paramétrica à biblioteca, agindo como um tradutor de coordenadas matemáticas brutas para diagnósticos de controle legíveis. A presença de um polo e um zero em distâncias infinitesimais altera drasticamente propriedades de controlabilidade e observabilidade do sistema. O módulo diagnostica matematicamente esses cenários através de laudos automáticos baseados na Teoria Analítica.

Na etapa algorítmica, as funções `_group_roots` e `_detect_cancellations` realizam uma varredura euclidiana no conjunto de polos e zeros encontrados. Se a distância métrica $\sqrt{(\text{Re}_p - \text{Re}_z)^2 + (\text{Im}_p - \text{Im}_z)^2} \leq \epsilon$ para uma tolerância rigorosa pré-definida, o sistema acusa o cancelamento numérico. Em seguida, rotinas de verificação local injetam pontos de prova ($s = z + \epsilon$) nos polinômios isolados avaliados via Horner, computando os limites laterais da função de transferência perto da singularidade para atestar a estabilidade estrutural local.

Uma característica avançada introduzida neste sistema especialista é a **parametrização contínua da variável simbólica**. Reconhecendo que modelos migram de s (tempo contínuo, Laplace) para z (tempo discreto) ou genéricos x , as sub-rotinas acatam o símbolo como argumento. A formatação converte todas as formulações textuais $N(\cdot)/D(\cdot)$ baseadas nessa chave matricial. O módulo, desta forma, preenche a lacuna analítica documentando passo-a-passo a desconstrução matemática efetuada no núcleo duro.

Listing 2.5 – Auditoria de Polos, Zeros e Tolerância (`analysis_explainer.py`)

```
1 def _detect_cancellations(zero_groups, pole_groups, tol=1e-4):
2     cancellations = []
3     pole_remaining = [len(group["roots"]) for group in pole_groups]
4     zero_remaining = [len(group["roots"]) for group in zero_groups]
5
6     for i, zero_group in enumerate(zero_groups):
7         for j, pole_group in enumerate(pole_groups):
8             if abs(zero_group["center"] - pole_group["center"]) <= tol:
9                 amount = min(zero_remaining[i], pole_remaining[j])
10                if amount > 0:
11                    cancellations.append(
12                        {
13                            "root": (zero_group["center"] + pole_group
14                                ["center"]) / 2,
15                            "count": amount,
```

```

15         }
16     )
17     zero_remaining[i] -= amount
18     pole_remaining[j] -= amount
19     return cancellations

```

2.6 gif_tools.py: Geração Didática de GIF (completamente opcional)

O módulo `gif_tools.py` adiciona um diferencial pedagógico, provendo meios visuais explícitos para documentar o desenrolar das manipulações algébricas sistêmicas sobre os vetores de zeros e polos. Enquanto o engenheiro observa o gráfico numérico resultante, o estudante em fase inicial necessita acompanhar a cinemática descritiva dos teoremas aplicados e as implicações de cada avaliação vetorial até a convergência final.

O motor implementa um laço iterativo fazendo forte uso da suíte de processamento de imagens `Pillow`. Elementos base do pacote, como as classes `ImageDraw` e `ImageFont`, são operados para gerar *frames* dinâmicos em memória RAM em altíssima qualidade *anti-aliased*. O desafio textual e espacial (*text-wrapping*) foi solucionado pela rotina matricial customizada de quebra de sentenças que garante margens consistentes conforme os laudos matemáticos complexos emitidos pelo `analysis_explainer.py` aumentam em profundidade.

Destaca-se também a rotina `create_root_locus_gif`, capaz de instanciar sistemas fictícios dinamicamente (*stubs*) em pleno processamento numérico para espelhar as raízes extraídas em vetores temporários a cada avanço escalar do parâmetro compensador K . O sub-módulo amarra consecutivamente centenas de *arrays* de pixels e escreve sequencialmente o fluxo animado, embutindo a referida notação paramétrica para coesão teórica ($G(z)$, por exemplo).

Listing 2.6 – Orquestração Numérica e Desenho de Imagem via Pillow (`gif_tools.py`)

```

1 def create_calculation_gif(tf, output_path, title="Processo de C lculo
  ", size=(960, 560), duration=900, variable_symbol="s"):
2     """Cria um GIF textual com os passos centrais do c lculo."""
3     steps = build_gif_steps(tf, variable_symbol=variable_symbol)
4     font_title = _load_font(34)
5     font_body = _load_font(24)
6     font_small = _load_font(18)
7
8     frames = []
9     for index, step in enumerate(steps):
10         image = Image.new("RGB", size, color=(18, 28, 38))
11         draw = ImageDraw.Draw(image)

```

```

12
13     # Omissao de detalhes esteticos de renderizacao por brevidade
14     ...
15     wrapped = _wrap_text(draw, step, font_body, size[0] - 120)
16     y = 208
17     for line in wrapped:
18         draw.text((60, y), line, fill=(245, 245, 245), font=
19             font_body)
20         y += 34
21
22     frames.append(image)
23
24 frames[0].save(
25     output_path,
26     save_all=True,
27     append_images=frames[1:],
28     duration=duration,
29     loop=0,
30     disposal=2,
31 )

```

2.7 time_response.py: Validação Temporal Fundamental

Após o mapeamento e validação das raízes singulares, o módulo `time_response.py` oferece o terreno para a constatação real do fenômeno físico no domínio do tempo (transiente). A teoria clássica de validação emprega sinais em degrau ou impulso unitário para expor o grau de subamortecimento previamente sugerido pela alocação vetorial das raízes de malha fechada computadas. Como uma característica fundamental de sua concepção pura, a biblioteca não utiliza o método analítico da Transformada Inversa de Laplace em sua base simbólica.

A resolução transiente é arquitetada sobre a integração diferencial pelo método numérico recursivo de Runge-Kutta de 4ª ordem (RK4). O algoritmo extrai diretamente da Função de Transferência suas matrizes A, B, C, D correspondentes ao espaço de estados controlável. Dado um vetor de passo de amostragem microscópico Δt , o passo preditor avalia quatro estágios derivativos intrínsecos (k_1, k_2, k_3, k_4) pesados assimetricamente no tempo. Este paradigma resolve efetivamente matrizes complexas, assegurando extrema precisão transiente e limitando estritamente os desvios causados por polos afastados na vizinhança de alta rigidez do sistema de EDOs.

Para injeções singulares como o *Delta de Dirac* (impulso iterativo), o cenário computacional exige uma modelagem discreta equivalente de densidade infinita e largura zero. O módulo instaura um retângulo perfeito limitando-se unicamente ao primeiro passo analítico ($n = 0$), possuindo largura baseada num *tick* Δt e magnitude ajustada de forma

rigidamente inversamente proporcional $u = 1/\Delta t$. O volume de energia provocado corresponde à exata injeção unitária predita pela teoria de sinais e sistemas, promovendo uma excitação natural da resposta ao estado original da matriz do sistema.

Listing 2.7 – Passo Preditor de Estado RK4 (`time_response.py`)

```

1 def _rk4_step(A, B, C, D, state, input_value, dt):
2     def f(current_state):
3         Ax = _mat_vec_mul(A, current_state)
4         Bu = [row[0] * input_value for row in B]
5         return _vec_add(Ax, Bu)
6
7     k1 = f(state)
8     k2 = f(_vec_add(state, _vec_scale(k1, dt / 2.0)))
9     k3 = f(_vec_add(state, _vec_scale(k2, dt / 2.0)))
10    k4 = f(_vec_add(state, _vec_scale(k3, dt)))
11
12    increment = _vec_scale(
13        _vec_add(
14            _vec_add(k1, _vec_scale(_vec_add(k2, k3), 2.0)),
15            k4,
16        ),
17        dt / 6.0,
18    )
19    next_state = _vec_add(state, increment)
20
21    y = sum(C[0][i] * next_state[i] for i in range(len(next_state))) +
22        D[0][0] * input_value
23    return next_state, y

```

2.8 `input_response.py`: Rastreamento Físico Arbitrário

Expandindo rigorosamente o potencial simulatório fundamental, `input_response.py` desvincula o comportamento de plantas LIT (Lineares, Invariantes no Tempo) de seus excitadores constantes. Enquanto a estabilidade global é propriedade única da topologia de seus polos — incondicional ao seu sinal ativador — em cenários pragmáticos, como referências variantes oscilatórias ou trajetórias polinomiais em rampa de erro estacionário, requer-se o acoplamento de avaliações não constantes.

Baseado na mesmíssima estrutura subjacente (o motor preditor `_rk4_step`), a função primária `arbitrary_input_response` absorve funções arbitrárias dinâmicas *callable* (por exemplo $\lambda(t) : t^2$) ou vetores finitos previamente discretizados. A cada iteração progressiva do laço temporal global delimitado entre $[0, T_{max}]$, o algoritmo sonda ativamente o valor equivalente do ruído exógeno instantâneo $U[k]$ e o provê como alimentação cruzada à malha.

A arquitetura segmentada desta funcionalidade externa realça a solidez e modularidade da biblioteca. Ao isolar conceitualmente a fonte geradora e mantendo as sub-equações do RK4 universais a *inputs* instantâneos escalares *u*, a lógica corrobora a definição canônica da decomposição da resposta de sistemas LIT: onde as respostas transientes inerentes fundem-se à manifestação dependente do sinal forçante, gerando *arrays* de saída (*y_values*) computáveis unicamente no núcleo sem qualquer tipo de colapso analítico-simbólico secundário.

Listing 2.8 – O Laço Numérico Extensível (input_response.py)

```

1 def arbitrary_input_response(tf, t_max, dt, input_signal):
2     """Simula a resposta para uma entrada arbitrária."""
3     if not isinstance(tf, TransferFunction):
4         raise TypeError("tf deve ser uma instância de TransferFunction")
5     if t_max <= 0 or dt <= 0:
6         raise ValueError("t_max e dt devem ser positivos.")
7
8     A, B, C, D = tf.to_state_space()
9     order = len(A)
10    state = [0.0] * order
11    t_values = []
12    y_values = []
13
14    if callable(input_signal):
15        def get_input(time_value, step):
16            return float(input_signal(time_value))
17    else:
18        samples = list(input_signal)
19        def get_input(_time_value, step):
20            return float(samples[step]) if step < len(samples) else 0.0
21
22    steps = int(round(t_max / dt))
23    for step in range(steps + 1):
24        current_time = step * dt
25        u = get_input(current_time, step)
26        t_values.append(current_time)
27        if step == 0:
28            y = sum(C[0][i] * state[i] for i in range(order)) + D[0][0]
29                * u
30            y_values.append(y)
31        else:
32            state, y = _rk4_step(A, B, C, D, state, u, dt)
33            y_values.append(y)
34    return t_values, y_values

```

2.9 gui.py: Interface de Operação Baseada em Eventos

Consolidando todos os módulos do ecossistema e fornecendo orquestração orientada a objetos de alta resiliência, o módulo `gui.py` gerencia a experiência humana na ferramenta sob a infraestrutura responsiva `Tkinter`. A sua implementação encapsula a suíte em formulários isolados que exigem manipulação extremamente veloz nas propriedades essenciais da malha (numerador vetorial e denominadores de graus). Seu *design* prioriza a acessibilidade tática, eliminando iterações onerosas em terminal isolado e possibilitando iterações cíclicas rápidas que definem a excelência no projeto clássico de compensadores em malha.

A arquitetura do fluxo aborda um desafio fundamental de sanitização de segurança sem abdicar de sintaxe simplificada e interpretativa a nível usuário. Para mitigar ambiguidades de conversão *string*-objeto intrínsecas a matrizes fornecidas diretamente por texto (`[x, y, z]`), a leitura aplica uma barreira de segurança formal baseada em blocos de `try-except` integrados com avaliadores abstratos léxicos primários como `'ast.literal_eval'`. Expressões puramente algébricas geradoras como compreensões de array iterativas baseadas na biblioteca padrão subjacente garantem maleabilidade na inserção, como a matriz vetorial do limiar de Ganhos K .

Essencialmente, ao acoplar as variáveis do usuário às constantes dos módulos geradores paramétricos, a tela expõe a maleabilidade do núcleo de simulação ao dispor as configurações da *Janela Limiar Focal* (Zoom de mapa com tolerância lateral definida para inspeção de agrupamentos) em união ao *slider* virtual do Rastreamento do Fator Crítico ζ . Cada invocação disparam instâncias do objeto `TransferFunction` processadas no *backend* do ecossistema acoplados perfeitamente a chamadas uníssonas nos canais de visualização via rotinas diretas na biblioteca isolada, mantendo o código primário intocado.

Listing 2.9 – Sanitização e Conversão Segura de Expressões Interpretadas (`gui.py`)

```

1  def _parse_list_or_expr(self, value):
2      try:
3          parsed = ast.literal_eval(value)
4          return list(parsed) if not isinstance(parsed, list) else
           parsed
5      except Exception:
6          return list(eval(value, {"__builtins__": {}}, {"range":
           range, "x": 0}))
7
8  def _parse_complex_value(self, value):
9      try:
10         parsed = ast.literal_eval(value)
11     except Exception:
12         parsed = eval(value, {"__builtins__": {}}, {"j": 1j, "
```

```
        complex": complex})  
13  
14     if isinstance(parsed, (int, float, complex)):  
15         return complex(parsed)  
16     raise ValueError("O centro do zoom deve ser um número real ou  
        complexo.")
```

3 Implementação e Validação

A eficácia de um sistema computacional de análise de sistemas lineares não reside apenas na precisão de seus algoritmos numéricos, mas na capacidade de integrar estes métodos em um fluxo de trabalho (workflow) intuitivo e focado. Este capítulo detalha a arquitetura operacional da aplicação, reestruturada especificamente para a investigação topológica de singularidades (polos e zeros). Discutem-se o ambiente de validação visual e os rigorosos mecanismos de *defensive programming* que protegem o usuário contra ambiguidades de entrada ou instabilidades numéricas durante a avaliação espacial.

3.1 Fluxo de Operação e Ciclo de Vida do Sistema

O sistema foi concebido para seguir o paradigma de **Validação e Diagnóstico Imediato**. Em oposição a suítes de análise multifuncionais que sobrecarregam o usuário com dados não solicitados, a arquitetura atual direciona a capacidade computacional para uma dissecação estrutural síncrona logo após a inserção do modelo.

Ciclo de Operação Didática e Estrutural

1. **Definição Simbólica:** O usuário parametriza a variável de domínio (ex: s, z ou x). O sistema absorve essa chave e a propaga globalmente, assegurando que laudos textuais e exportações gráficas mantenham a coerência teórica adequada ao contexto (contínuo ou discreto).
2. **Input e Sanitização Segura:** As matrizes polinomiais são processadas por um *parser* rigoroso (`ast.literal_eval`). Etapas de *trimming* (remoção de ruído numérico de ponto flutuante) e normalização (forçando polinômios mônicos) são aplicadas antes da instância formal.
3. **Motor de Diagnóstico Especialista:** Antes da geração de qualquer gráfico, o subsistema `analysis_explainer` realiza uma varredura heurística. Ele identifica múltiplos, agrupa raízes de alta proximidade e detecta quase-cancelamentos dinâmicos, emitindo um laudo textual que orienta a expectativa do usuário.
4. **Renderização Focal sob Demanda:** Os gráficos de lugar geométrico e mapas espaciais são renderizados respeitando os limites paramétricos definidos pela interface (como a janela de "Zoom" e o ângulo limite do amortecimento ζ).

3.2 Galeria de Funcionalidades e Validação Visual

Para atestar a coesão entre o motor matemático e a experiência de uso, a seguir apresentam-se os estados operacionais primários da interface. As capturas documentam a capacidade do sistema em mapear o espaço complexo com precisão, validando os cálculos globais de Durand-Kerner através da suíte de renderização isométrica.



Figura 1 – Interface de Entrada e Diagnóstico: O sistema processa matrizes de forma segura e exibe imediatamente o passo a passo da extração estrutural, destacando cancelamentos locais e ordens de multiplicidade antes da visualização geométrica.

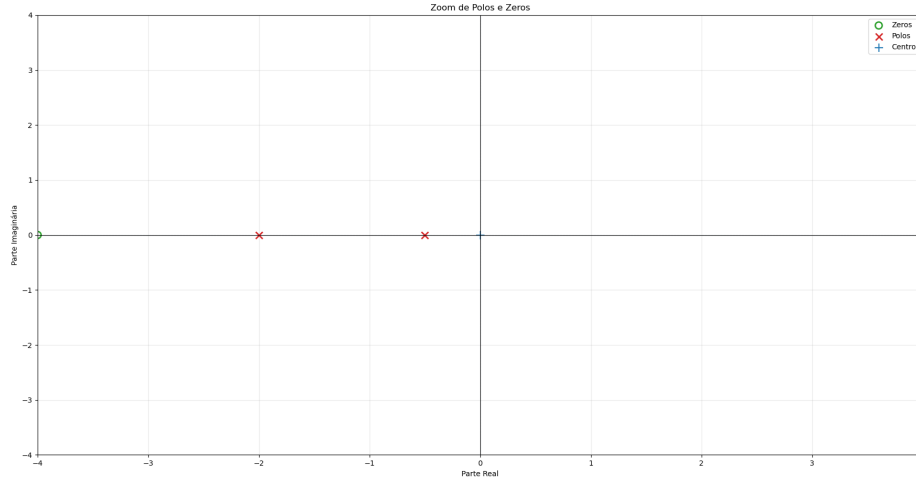


Figura 2 – Mapa Global de Polos e Zeros: Visão macroscópica das fronteiras sistêmicas. A isometria garante que distâncias euclidianas no plano complexo não sofram distorções visuais de escala.

3.3 Processo de Robustez

Para que a ferramenta seja confiável em ambientes de pesquisa e ensino elucidador, é indispensável que o ambiente resista a inserções irrealistas sem encerramentos abruptos. A arquitetura de *defensive programming* opera com validações físicas e topológicas estritas:

- **Critério de Quase-Cancelamento e Tolerância Euclideana:** Durante a auditoria do laudo textual, agrupamentos de raízes utilizam uma margem $\epsilon = 10^{-4}$ (parametrizável). Caso a distância vetorial no plano $|p - z|$ caia nesse limiar, o sistema avisa proativamente o usuário sobre o cancelamento numérico em vez de propagar instabilidades transientes nulas para as malhas seguintes.
- **Verificação de Propriedade Estrita (Causalidade):** Antes de autorizar a conversão matricial para o Espaço de Estados (`to_state_space`), necessária ao avanço iterativo do Runge-Kutta, o código atesta se o grau do polinômio numerador ultrapassa o denominador. Plantas não próprias são mecanicamente rejeitadas com avisos elucidativos, prevenindo a injeção de derivadas puras implícitas nos vetores físicos temporais.
- **Garantia Estrutural no LGR iterativo:** No traçado do Lugar das Raízes, ao invés de aproximar ramificações por cálculos assintóticos (sujeitos a quebras visuais complexas), o código insere uma matriz iterável de ganhos K diretamente no refinador global `polynomial_roots`. Isso garante a fidelidade de bifurcações e de intersecções com a reta do fator de amortecimento ζ , impedindo estimativas errôneas de ganho crítico em topologias densas.

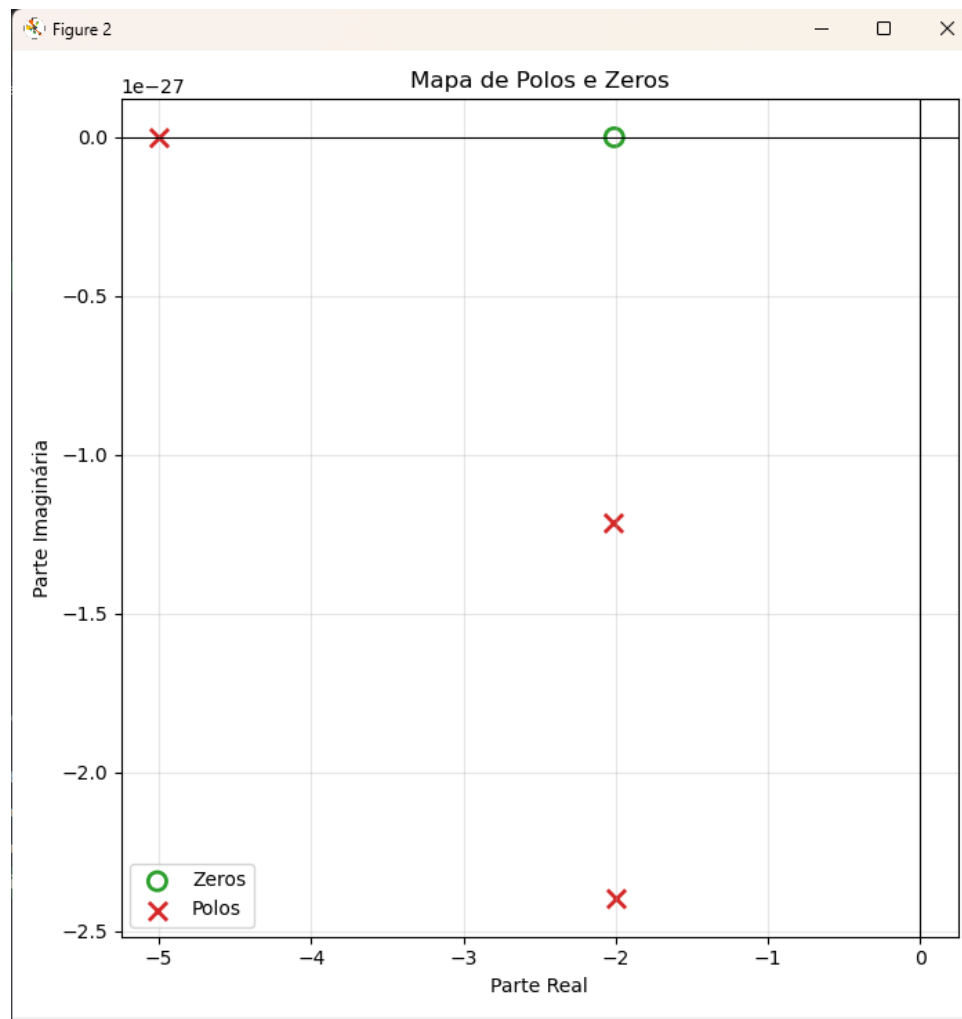


Figura 3 – Janela de Inspeção Microscópica (Zoom): Ferramenta visual vital para validação de polos múltiplos e dinâmicas de quase-cancelamento que definem o comportamento transitório oculto da planta em baixas frequências.

- **Delimitação Trigonométrica no Plano:** O algoritmo que insere o limite paramétrico de controle ζ isola rigorosamente a entrada na faixa estrita de $-1 < \zeta < 1$. Valores fora do contradomínio das funções arco-cosseno (**acos**) gerariam interrupções drásticas de *software*; a normalização defensiva restringe os feixes diretivos mantendo a janela de plotagem segura e ininterrupta.

Esta blindagem algorítmica garante que a biblioteca funcione não apenas como uma calculadora estática, mas como um simulador inteligente capaz de guiar e corrigir as formulações teóricas do acadêmico antes do processamento intensivo.

Para ilustrar a versatilidade geométrica do sistema frente a diferentes topologias matemáticas, esta seção apresenta uma galeria complementar com recortes menores e detalhamentos específicos. As imagens evidenciam a capacidade do motor de lidar com variações abruptas de amortecimento e configurações densas no plano complexo.

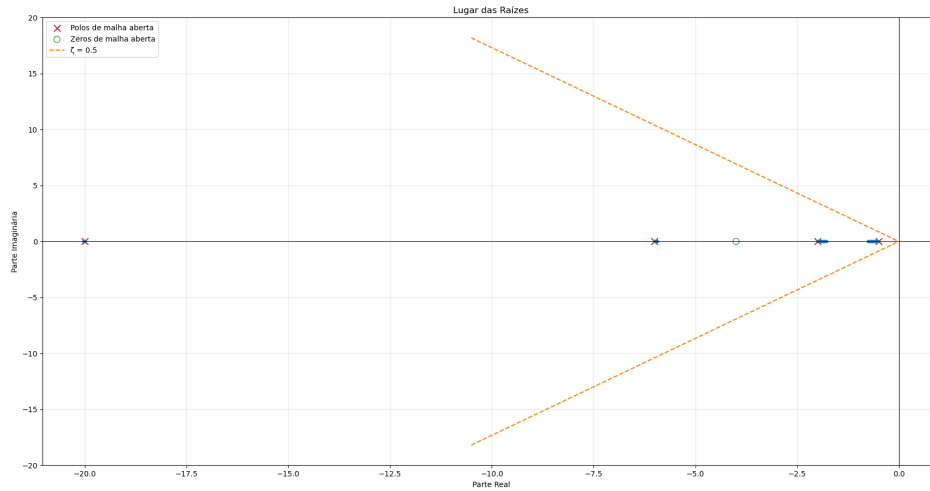


Figura 4 – Lugar das Raízes com Restrição de Amortecimento (ζ): Validação visual da evolução dinâmica sob perturbação de ganho paramétrico K . A interseção com a linha de ζ indica o ganho exato para a conformidade de transiente exigida pelo projeto.

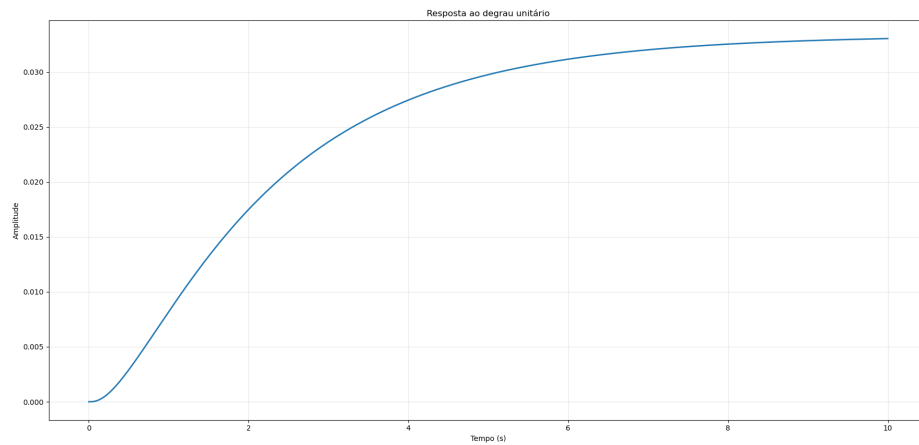


Figura 5 – Validação Temporal Transiente: Integrado como módulo de suporte empírico, atesta que o sobre-sinal físico da planta simulada corresponde exatamente à previsão geométrica da topologia dos polos.

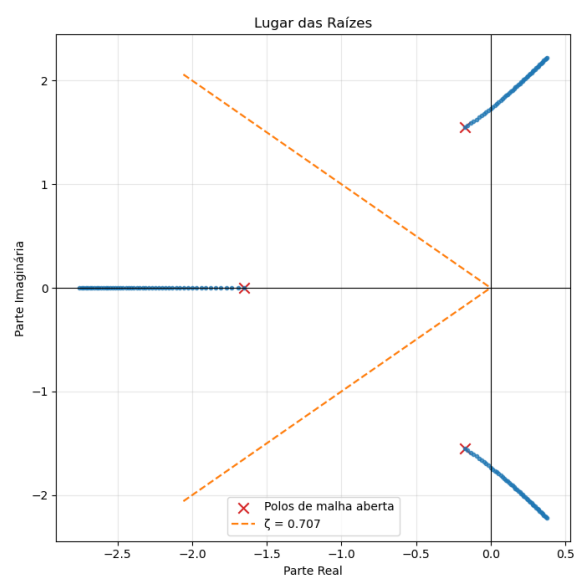


Figura 6 – Variação Paramétrica: Ajuste da restrição de amortecimento para $\zeta = 0.707$, ilustrando a abertura do ângulo limite.

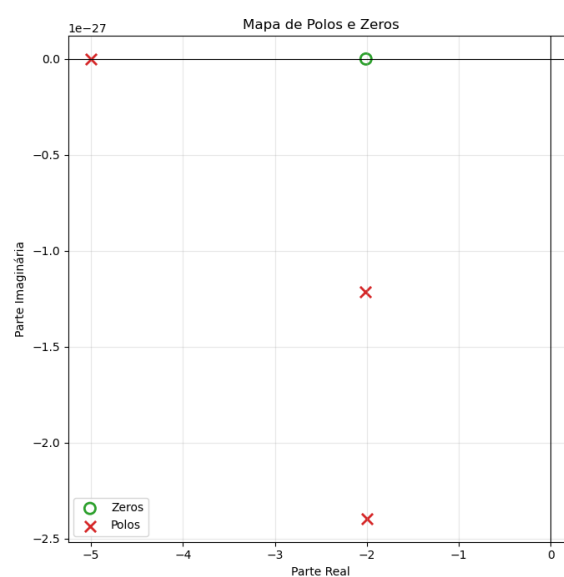


Figura 7 – Aglomerado Singular: Detalhe microscópico evidenciando polos de alta multiplicidade operando no limite da estabilidade.

4 Conclusão

A realização deste trabalho resultou no desenvolvimento, validação e consolidação de uma arquitetura computacional inteiramente dedicada à análise estrutural de Sistemas Lineares e Invariantes no Tempo (LIT). Afastando-se de abordagens genéricas em ferramentas comerciais de amplo espectro, o sistema provou-se altamente eficaz ao focar seu núcleo de processamento na extração exata das propriedades intrínsecas da planta, notadamente a localização de polos e zeros e a avaliação da estabilidade através do Lugar Geométrico das Raízes parametrizado pelo fator de amortecimento (ζ).

A implementação do núcleo algébrico nativo, fundamentado no método iterativo global de Durand-Kerner, demonstrou ser uma escolha arquitetural robusta. O sistema foi capaz de contornar a dependência de bibliotecas numéricas pré-compiladas na formulação de base, garantindo ao operador um controle absoluto sobre tolerâncias computacionais e precisão na determinação de raízes complexas conjugadas e aglomerados de alta multiplicidade. A convergência rigorosa entre as abstrações polinomiais simbólicas e a conversão para a forma canônica controlável no espaço de estados validou a integridade da ponte matemática desenvolvida entre o domínio complexo e as simulações de validação transitente no tempo.

A interface visual isométrica e as ferramentas de análise microscópica mostraram-se fundamentais para a superação de ambiguidades teóricas. A implementação da janela de *Zoom* paramétrico e o rastreamento rigoroso da interseção de raízes com os feixes restritivos de amortecimento (ζ) permitiram uma visualização livre das distorções comuns em esboços assintóticos. Adicionalmente, a estruturação de uma camada de *defensive programming* atuou com sucesso como um sistema especialista: ao prever quase-cancelamentos por tolerância euclidiana e auditar propriedades de causalidade rigorosamente, a aplicação mitigou a propagação de instabilidades físicas irreais.

Por fim, a metodologia de projeto de *software* adotada mostrou-se bem-sucedida ao integrar o mais alto rigor matemático a um ambiente pedagógico iterativo, livre do excesso de atrito comum em ambientes orientados a terminal. A capacidade de gerar laudos diagnósticos em tempo real e de exportar o passo a passo da desconstrução algébrica através de mídias dinâmicas consolida o programa não apenas como um motor de cálculo, mas como um autêntico instrumento didático. Com a validação total deste motor estrutural, o projeto estabelece uma fundação algorítmica sólida, perfeitamente apta para futuras expansões focadas na síntese automática e no design de controladores clássicos em malha fechada.

Referências

DORF, R. C.; BISHOP, R. H. *Sistemas de Controle Moderno*. 12. ed. Rio de Janeiro: LTC, 2011.

GOLNARAGHI, F.; KUO, B. C. *Automatic Control Systems*. 9. ed. New York: John Wiley & Sons, 2010.

NISE, N. S. *Engenharia de Sistemas de Controle*. 6. ed. Rio de Janeiro: LTC, 2012.

OGATA, K. *Engenharia de Controle Moderno*. 5. ed. São Paulo: Pearson Prentice Hall, 2010.